

Hybrid Adversarial Retraining Approach for a Proactive Network Intrusion Detection System

NOORA AL ROKEN^{1,2}, HAKIM HACID³ (Senior Member, IEEE), AHMED BOURIDANE², AND ABIR HUSSAIN¹ (Senior Member, IEEE)

¹Department of Electrical Engineering, University of Sharjah, Sharjah, United Arab Emirates

²Department of Computer Engineering, University of Sharjah, Sharjah, United Arab Emirates

³Technology Innovation Institute, Abu Dhabi, United Arab Emirates

CORRESPONDING AUTHORS: N. A. ROKEN (u22105839@sharjah.ac.ae) AND A. HUSSAIN (abir.hussain@sharjah.ac.ae)

ABSTRACT Artificial intelligence provides benefits across many areas, including network security. It also enables novel attacks and new security risks. However, AI-generated attacks now outpace the capabilities of current defense systems. Therefore, it is crucial to implement proactive security measures to identify unknown threats and protect digital systems. Existing research is limited by small or outdated datasets, data from virtual machines, inefficient or complex behavior-based methods, and reliance on a single adversarial condition. To address this, our study developed a new attack-detection framework. We used a proactive approach with a hybrid retraining technique that integrates both black- and white-box attacks. The framework combines two different attack concealment methods with a two-stage classification method to enhance detection accuracy and reduce false negatives. We joined two large network attack datasets, CICIDS2017 and CSE-CIC-IDS2018, to train and test the framework. Experiments show that our intrusion detection system achieved high detection rates. The two-layer convolutional neural network, using our all-attacks adversarial retraining method, demonstrated superior generalization across 12 adversarial attack algorithms, achieving an overall accuracy of 89%. This finding shows that simpler models can still provide robust protection with our hybrid retraining approach. Our method improved accuracy by up to 7.02% on CICIDS2017 and 3.7% on CSE-CIC-IDS2018. These results demonstrate that our method provides a practical and reliable defense against real-world network security threats.

INDEX TERMS Adversarial attacks, adversarial learning, network attacks, network intrusion detection, network security.

I. INTRODUCTION

IN THE current time, securing computer systems has become crucial, as organizations have undergone digital transformations, making them more vulnerable to network attacks and malware. In addition, they have also become a potential target for evolving cybersecurity threats, such as adversarial attacks. According to the 2025 cyberthreat defense report [1], on a scale of 1-5, organizations have indicated a lack of security awareness, security personnel, and burden of analyzing big data with scale values of 3.55, 3.55, and 3.44, respectively. This data suggests that organizations continue to face challenges in maintaining an effective defense system. With the emergence of adversarial attacks and mutation engines, a limitation has arisen with the traditional rule-based Network Intrusion Detection Systems (NIDS). These traditional approaches are inefficient in addressing the rapid evolution of meticulously crafted

evasion attacks, which can modify their code and/or structure to evade detection by reactive defense mechanisms. Based on the survey results in [1], cybersecurity analysts believe that notorious generative AI applications, such as WormGPT, FraudGPT, and DarkBART, will enhance vulnerability detection and increase attack sophistication. Attackers can exploit AI to produce sophisticated attacks in large volumes and at increased speeds.

Another report indicates that 96.4% of respondents believe that cybercriminals will benefit from AI, while 96.9% believe it will assist IT security personnel in defending against attacks [2]. The progress in AI has yielded numerous advantages and prospects across various domains, including network security. AI provides robust capabilities and tools to strengthen defense systems and enhance security measures. It can process vast quantities of data, recognize patterns, and spot anomalies at a pace and scale beyond human capabilities.

Therefore, the key challenge is to develop proactive security defenses capable of detecting both known and unknown attacks before deployment, and to construct safer digital systems that automatically adapt to the rapidly evolving threats.

This paper examines various network attacks and their sophisticated variants that can execute malware scripts after successfully exploiting a vulnerability, through extensive experiments to identify the most effective AI-based NIDS for detecting six attack types, including adversarial variants. According to [2], an attack can disrupt, deny, degrade, or destroy a system for the duration of its execution, and it can carry malware that extends its capabilities by executing various commands, such as wiping databases, recording communications, or stealing credentials. While prior studies evaluate robustness against a limited number of adversarial techniques or focus on either white-box or black-box settings in isolation, this work distinguishes itself by integrating 12 adversarial attack algorithms (i.e., six white-box and six black-box) within a unified training and evaluation framework. Considering only one adversarial attack condition or a few algorithms is insufficient for attack detection deployment, since the model has a narrower view of the adversarial attack variants. Meanwhile, combining both white- and black-box attack algorithms expands the spectrum of seen attack conditions, which enhances the generalization of the model, making it more reliable to implement in real-world scenarios to detect unseen attack variants.

The key novelties of this study are as follows:

- The Implementation of six white-box and six black-box adversarial attack algorithms across six attack categories on a large, combined dataset to generate adversarial samples was performed to generate adversarial samples and enhance model robustness through adversarial retraining.
- The development of a comprehensive proactive AI-based NIDS model achieved superior performance compared to existing methods, demonstrated generalization, and represented the most comprehensive framework proposed to date.
- We demonstrate that simpler architectures can achieve high performance, reducing the need for computationally complex models.

The rest of the paper is structured as follows: Section II presents a review of related works. Section III details the methodology, including system overview, data preprocessing, feature and model selection, and adversarial retraining approach. Finally, Section IV-C presents the results, and the paper concludes with Section V.

II. LITERATURE REVIEW

Zhang et al. [3] introduced the Transferred Generative Adversarial Network-Intrusion Detection System (TGAN-IDS), which uses transfer learning by utilizing the pre-trained Deep Convolutional GAN (DCGAN) to train the generator for producing realistic adversarial samples, and the PreD

Convolutional Neural Network (CNN) to train the discriminator for binary attack classification. The GAN is trained until the discriminator can effectively classify both known and unknown samples. TGAN-IDS achieved a 98.70% F1-score in binary classification on the KDD99 dataset [4]; however, it is important to note that this dataset is outdated, having been created in 1999. Heydari and Nyrao [5] presented an adversarially trained Artificial Neural Network (ANN) and compared its performance with that of a transformer-based neural network, a Multi-Layer Perceptron (MLP), and a Random Forest (RF). Although their model achieved 85% accuracy against 0.01 perturbations from Projected Gradient Descent (PGD), Fast Gradient Sign Method (FGSM), and FGSM-based black-box adversarial attacks, this accuracy is considered relatively low, particularly given the less sophisticated nature of the white-box attacks.

In [6], the authors used a CNN-based NIDS trained with PGD attacks and the Pigeon-Inspired Optimization Algorithm (PIOA). The PIOA provides optimal perturbation directions, while the PGD maintains the perturbation extent. The testing samples are generated using Jacobian Saliency Map Attacks (JSMA) and are smoothed using Spatial Smoothing (SS) technique. The results were validated on the CIC-DDoS2019 [7], which contains only Distributed Denial of Service (DDoS) attacks, and on UNSW-NB15 [8], which includes four attack types. The results demonstrated approximately 99% accuracy across both datasets; however, the model was tested only against the FGSM attack, a less sophisticated attack, suggesting susceptibility to more advanced black-box attacks. Additionally, white-box attacks are unrealistic, as it is rare for an attacker to gain insights into the defense model architecture. Furthermore, their 5-layer CNN is highly complex, with each layer producing 60,000 activations, including the input layer.

The Strategic Feature-Adaptive Adversarial Attack (SFAA) algorithm is used in [9] to generate black-box adversarial samples through a Multi-Layer Perceptron (MLP) that incorporates gradient information while accounting for feature correlation. The behavior of the generated adversarial attacks is extracted and subsequently transferred to DNS over HTTPS (DoH) tunneling, enabling the malicious DoH packets to evade defense systems. Their defense model, Dual-Defense Adversarial Framework (DDAF), operates in two stages. The initial stage involves adversarial training, in which the generated adversarial samples are augmented into the original dataset. The subsequent stage employs a confidence-based decision method to re-evaluate any benignly classified sample that falls below the designated threshold. Although DDAF demonstrated high detection rate, its robustness is limited to DoH-like tunneling channels and does not extend to volumetric or behavioral attacks such as DoS, brute force, botnet, infiltration, or portscan, which are more prevalent and not specialized threats.

In [10], the authors propose the Strolling in Practical Traffic Space (SPTS) framework, which leverages feature hierarchy and rectification to modify features rather than

altering arbitrary random features. According to their findings, conventional adversarial training (i.e., training with adversarial samples generated without hierarchy and rectification) achieved 94.8% accuracy on only four black-box adversarial attacks, including the Diversity Gradient Method (DGM), Scale-Invariant Method (SIM), and two FGSM variants, when employing MLP as the surrogate model and a self-attention-based neural network as the target defense model. Utilizing their framework, they achieved an average accuracy of 95.7%, representing an approximate 1% increase.

Tafreshian and Zhang [11] employed ensemble methods to generate black-box Genetic Algorithm (GA) attacks and develop defense models. Although this black-box strategy is a reasonable choice for simulating real-world conditions, the study does not assess the effectiveness of the defensive framework against a broader range of adversarial attack techniques. The reliance on a GA attack may constrain the model robustness to other forms of adversarial perturbations. Furthermore, the work lacks details regarding the dataset splitting and evaluation methodology. The authors indicate the use of the Synthetic Minority Over-sampling Technique (SMOTE) to balance the dataset, yet it remains unclear whether this was applied solely to the training set or to the entire dataset, including the testing set. Applying SMOTE to the testing set could introduce performance bias, such as high accuracy.

An alternative approach proposed in [12] is the Adversarial Purification through Reconstruction (AdvPurRec) model, which utilizes an autoencoder to clean noisy samples that mimic adversarial behavior before they are processed by the MLP defense model. However, the Gaussian noise-perturbed samples used for training do not appear representative of adversarial samples, because in the testing set, only the Zero Order Optimization (ZOO) algorithm samples achieved high accuracy, while the remaining five testing algorithms were largely undetected.

Huang et al. [13] introduced the Contrastive Adversarial Representation Distillation (CARD) framework, which employs contrastive learning between a source and target model. In their framework, the source model is trained on a larger dataset, and both models are designed to distinguish between positive pairs (i.e., normal, adversarial, and naturally corrupted attack samples) and negative pairs (i.e., benign samples). The framework encourages the learning of similar representations for positive pairs and discriminative representations for negative pairs. The source model is particularly useful in scenarios of data scarcity and can be adapted for models with varying input and model dimensions through adaptive input and dimensionality alignment. Adversarial samples are generated using PGD, while naturally corrupted samples are produced by adding Gaussian noise. The authors reported achieving the highest adversarial accuracy of 81% and the highest corruption accuracy of 84%. However, the models they employ, including WideResNet-34-10 and ResNet-18, are computationally intensive. Additionally, the authors assume that the attacker has knowledge of the target

model and the adversarial attacks used to train it, a scenario that is rare in real-world applications.

The Perturb-ability Score (PS) was introduced in [14] where features were categorized into low, medium, and high, indicating their likelihood of being used for perturbation using two methods. The first method selected only low-PS features as model input, while the second method masked features during both training and testing or only during testing. The authors concluded that inference-only masking was the least resource-intensive defense approach, with minimal performance degradation. However, it demonstrated unstable performance, particularly with the Support Vector Machine (SVM) model, necessitating masking during training to understand behavior and maintain satisfactory performance.

Several research papers have focused on the development and evaluation of adversarial attacks, without considering the enhancement of defense mechanisms. Ali et al. [15] evaluated three white-box adversarial attacks on a CNN with Long-Short Term Memory (CNN-LSTM) model, showing that PGD samples exhibited the highest evasion rate, followed by FGSM and Carlini and Wagner (CW). An ANN-based GAN was introduced [16], utilizing an attention mechanism to select relevant features for modification and active learning to further adjust adversarial samples with probabilities close to 0.5. Additionally, a Particle Swarm Optimization (PSO) black-box algorithm was employed to generate adversarial samples in [17]. The authors reported a degradation in accuracy ranging from 40% to 80%; however, the KDD99 dataset utilized was outdated, while more recent attacks are currently in use.

Grini et al. [18] adopted an alternative methodology for presenting valid and invalid adversarial samples. The authors claim that the majority of white-box adversarial attacks generated by six algorithms are invalid, as they alter some or all features. However, they did not account for the fact that not all features are utilized in these attack algorithms since feature selection is performed. Additionally, Transfformer is introduced in [19] to generate adversarial samples. The authors employ feature selection, cluster continuous values, and a self-attention multihead generator to capture relevant sections and their dependencies for generating new samples. Another methodology is presented in [20], which uses eXplainable AI (XAI) to extract the most relevant independent features with the fewest False Negatives (FN) (i.e., misclassified attack instances). The authors use KernelSHAP in a black-box setting, without access to gradient details. They considered brute-force and Cross-Site Scripting (XSS) attacks and selected three independent features. For the brute-force attack, they set the number of Push (PSH) flags in the forward direction to 0, and for the XSS, they padded the Synchronize (SYN) and Acknowledge (ACK) packets. Their results demonstrated evasion rates of 95.65% and 100% for their Artificial Neural Network (ANN) model against brute-force and XSS attacks, respectively. However, the dataset is relatively small and unreliable, with approximately 8,000

TABLE 1. Related works summary.

Paper	Category			Dataset	IDS Training Type			Setting		Model
	Attack	Defense	Attack & Defense		Adversarial	GAN-Based	Other	White-Box	Black-Box	
[3]		✓		Malware-Traffic-Analysis.net, PacketTotal, Interactive Online Malware Analysis Sandbox, CICIDS2017		✓			✓	TGAN-IDS
[5]			✓	UNSW-NB15	✓			✓		MLP
[6]			✓	CIC-DDoS-2019, UNSW-NB15	✓			✓		CNN
[9]			✓	CIRA-CICDoHBrw-2020, DoH-Tunnel-Traffic-HKD, large-scale benign DoH data	✓				✓	KNN, SVM, RF, DT, XGBoost, CatBoost, LightGBM with SVM
[10]			✓	CSE-CICIDS2018, TON_IoT	✓				✓	MLP, CNN, ResCNN, LSTM, Self-attention
[11]			✓	NSL-KDD, UNSW-NB15	✓				✓	Ensemble: LR+RF, LSTM+MLP
[12]			✓	UNSW-NB15, CSE-CICIDS2018			✓	✓		MLP autoencoder
[13]			✓	NSL-KDD, UNSW-NB15	✓			✓		ResNet-18, MobileNet
[14]			✓	UNSW-NB15, CSE-CICIDS2018			✓	✓		MLP, SVM, RF, CNN
[15]	✓			CICIDS2017			✓	✓		CNN-LSTM
[16]	✓			CICIDS2017			✓		✓	MLP-based attention GAN
[17]	✓			KDD99			✓		✓	PSO
[18]	✓			NSL-KDD			✓		✓	MLP
[19]	✓			CTU13, CICIDS2017, BoT-IoT, ToN_IoT, Kitsune			✓		✓	Transfformer
[20]	✓			CICIDS2017, ToN_IoT			✓		✓	Kernel-SHAP
Prop.			✓	CICIDS2017, CICIDS2018	✓			✓	✓	CNN

samples for the File Transfer Protocol (FTP) brute-force class and 657 for the XSS class. The aforementioned studies did not propose any defense mechanisms. As summarized in Table 1, existing studies typically consider either white-box or black-box adversarial conditions independently. In contrast, the proposed model integrates both settings within a unified framework to enhance generalization, a combination that has not been explored in prior work.

III. METHODOLOGY

This section of the paper presents the threat model, overall system design, datasets used, pre-processing and feature selection, and candidate networks for the proposed NIDS. It explains the NIDS input, the extracted features, the attacks considered, and the hyperparameter tuning method used.

A. THREAT MODEL

In this study, we consider a threat model combining both white-box and black-box adversarial scenarios. In the white-box scenario, the adversary has comprehensive knowledge, including access to the entire NIDS configuration, which comprises the input, number of layers, hyperparameters, and outputs. The attacker generates adversarial attack instances, which are evaluated against the same NIDS, and uses those that evade detection. Conversely, in the black-box scenario, the attacker is only aware of the input and output, without

any details about the model configurations. In this context, the adversary generates adversarial samples using a different model and subsequently queries (i.e., sends) them to the actual NIDS until they evade detection. We have integrated both conditions to develop a generalized model. In both methods, the attacker can modify all selected input features except the destination port, since certain attacks depend on it. Furthermore, the objective of attack generation considered in this paper is targeted, aiming to minimize the distance between the original attack instance x and the perturbed attack instance x' while ensuring classification as the target benign class t , as presented in (1) [21]:

$$\min_{x'} d(x', x) \text{ s.t. } C(x') = t \quad (1)$$

B. SYSTEM DESIGN

The proposed system, illustrated in Fig. 1, comprises several phases. Initially, a NIDS is developed to identify known attacks only (i.e., a reactive approach). This study examines two NIDS scenarios, as shown in Fig. 2. The first case is a multiclass classification problem in which each attack is classified separately from the benign class. The second case is a two-stage classification: binary and multiclass. The first classifier distinguishes between benign and attack network flow packets, and the second stage detects which attack is about to occur. The rationale for testing different cases is to

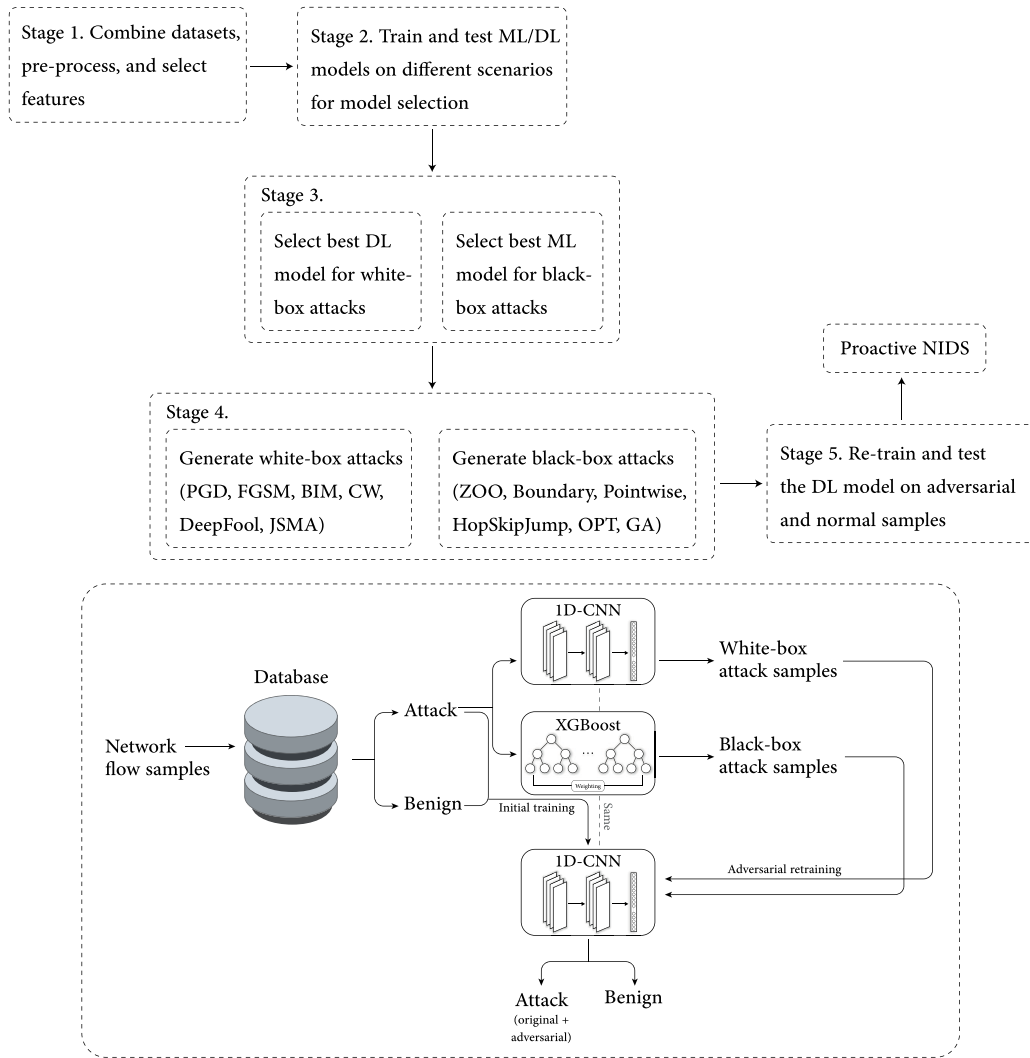


FIGURE 1. The proposed system design.

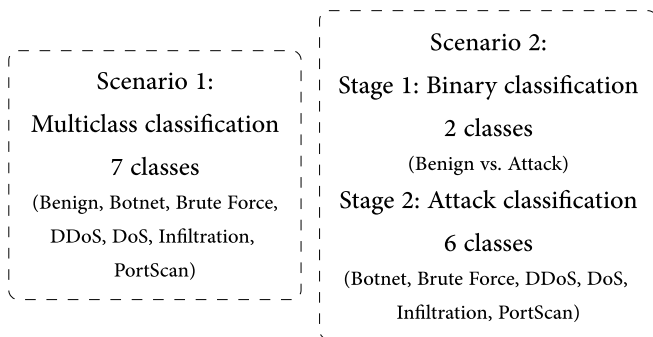


FIGURE 2. The considered NIDS scenarios for efficient attack detection.

select the most accurate method to correctly detect attacks and distinguish them from benign packets, thereby achieving an efficient NIDS.

Following the NIDS construction, six white-box attacks are generated: Projected Gradient Descent (PGD) [22], Fast Gradient Sign Method (FGSM) [23], Basic Iterative Method

(BIM) [24], Carlini & Wagner Attack (CW) [21], DeepFool [25], and Jacobian-based Saliency Map Attack (JSMA) [26]. These white-box attacks utilize the same initial NIDS architecture which uses a Deep Learning (DL) model, with successfully evaded samples reserved for retraining. Subsequently, black-box attacks are created using six existing algorithms: Zeroth Order Optimization (ZOO) [27], Boundary attack [28], Pointwise attack [29], HopSkipJumpAttack [30], OPT-Attack [31], and Genetic Algorithm (GA) [32]. As with white-box attacks, samples that successfully evade the NIDS are used for retraining. However, the black-box scenario implements a different NIDS that uses a Machine Learning (ML) model.

In the final phase, all collected samples are used to retrain the initial DL NIDS to enhance its robustness and proactivity (i.e., enabling detection of known and unknown attacks).

C. DATASET PRE-PROCESSING

Researchers at the Canadian Institute for Cybersecurity (CIC) at the University of Brunswick have created benchmark

datasets that capture real-world network traffic attacks exploiting commonly used protocols, with a large number of samples collected [33]. The CICIDS2017 [33] dataset comprises network traffic flows between victim and attacker networks. The victim network includes two switches, three servers, one firewall, and ten PCs, while the attacker network consists of one router, one switch, and four PCs. Additionally, the victim network incorporates all three commonly used operating systems, namely Windows, Linux-Ubuntu, and Macintosh, to ensure comprehensive coverage, whereas the attacker network employs Linux-Kali and Windows to generate the attacks. A mirror port is configured on the victim switch to capture all inbound and outbound traffic for analysis and anomaly detection. The second dataset used is the CSE-CIC-IDS2018 [34], an extended, larger dataset with 16 million samples, compared to the 2.8 million in CICIDS2017. While newer datasets are available, two primary constraints guided our selection. First, the dataset must contain attacks specifically targeting computer network intrusion detection systems, rather than other targets such as Internet of Things (IoT) devices, electric vehicles, or Domain Name System (DNS) server. Second, the dataset should be sufficiently populated and include a diverse range of attack types to ensure comprehensive evaluation. Based on these criteria, we selected the CICIDS2017 and CSE-CIC-IDS2018 datasets.

Prior to combining the datasets, uncommon features were removed from the CSE-CIC-IDS2018 dataset, specifically protocol and timestamp features, as well as flow ID, source IP, source port, and destination IP, for the “Tuesday-20-02-2018_TrafficForML_CICFlow_Meter.csv” file only. The number of initial bytes sent in the forward and backward directions was removed because it contained unrealistic negative values in most instances. Additionally, 2,989 and 131,799 NA instances were removed from the number of bytes and packets sent per second for the CICIDS2017 and CSE-CIC-IDS2018 datasets, respectively. In the CICIDS2017 dataset, the forward header length feature was duplicated, necessitating the removal of one of them. The final dataset comprised 76 features, including the label. Furthermore, variants of each attack were grouped into a single class. For instance, Denial of Service (DoS) GoldenEye, DoS Hulk, DoS SlowHTTPTest, and DoS Slowloris were combined into the DoS class. As illustrated in Fig 3, the attack dataset is highly imbalanced, with the majority class (i.e., DDoS) containing approximately 1.4 million samples, while the minority class (i.e., heartbleed) contains only 11 samples. Consequently, heartbleed, Patator, and web attacks were removed, and the remaining attack classes, along with the benign class, were undersampled. Figure 4 indicates that the majority of captured samples were benign, with 15.7 million instances. The final set of attacks considered in the datasets included DoS, DDoS, brute-force, portscan, botnet, and infiltration, which exploited the HTTP, HTTPS, SSH, FTP, and email protocols. Undersampling was conducted randomly based on the number of samples in the minority

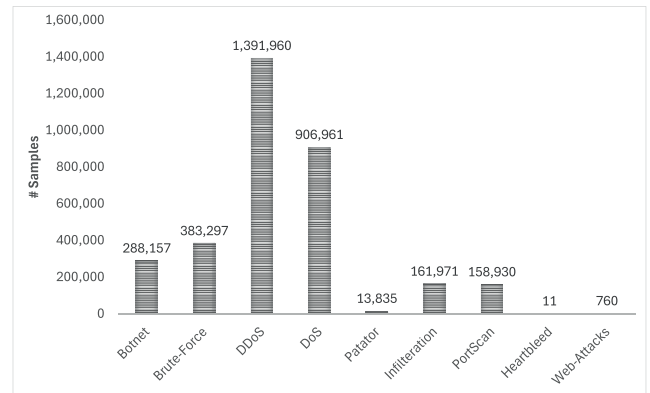


FIGURE 3. Number of samples per attack class.

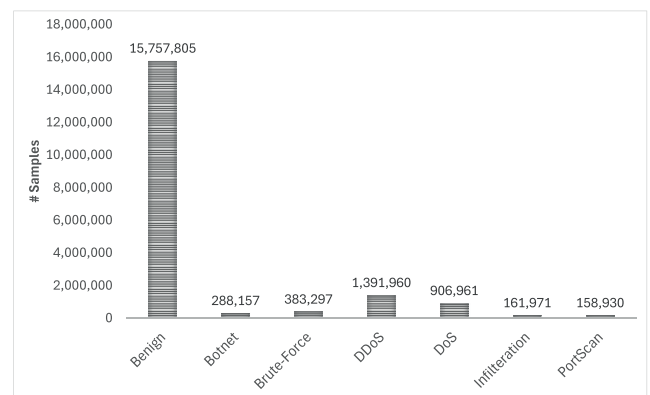


FIGURE 4. Number of benign and attack samples before undersampling.

classes, infiltration, and portscan. The undersampling was carried before the training and testing split and the “RAND()” MySQL function was used to select the instances. For feature and model selection, we have generated three random subsets and ran the algorithms on them. The results were very similar and hence, we have proceeded with using one subset.

As shown in Table 2, in the initial scenario, both the attack and benign classes were undersampled to approximately 160,000 random samples. Conversely, in scenario 2.1 of Table 2, the attack classes were grouped into a single class, resulting in approximately 3.3 million samples for both the benign and attack classes. In scenario 2.2, the multiclass dataset was used, except for the benign class. To evaluate the models, an 80:20 split was used instead of cross-validation, since the datasets are sufficiently large to assess them, as shown in Table 2. Furthermore, for comparison with existing studies, we have independently preprocessed both datasets, with the number of samples detailed in Table 2. The feature and model selection, as well as adversarial retraining, will utilize the same configuration for the benchmark datasets based on the combined dataset outcome.

D. FEATURE SELECTION

In contrast to the approach taken in [33], where feature selection was applied to each class independently using

TABLE 2. Training and testing sample distribution for multiclass and binary classification.

#	Reason	Dataset name	Type	Classes	# Samples	Training (80%)	Testing (20%)
1		CICIDS	Scenario 1: Multiclass	Benign	160,785	128,628	32,157
				Bot	160,147	128,118	32,029
				Brute-force	160,784	128,627	32,157
				DDoS	160,939	128,751	32,188
				DoS	160,848	128,678	32,170
				Infiltration	160,674	128,539	32,135
				Portscan	158,804	127,043	31,761
				Total	1,122,981	898,384	224,597
2	Combined dataset for proposed model	CICIDS_Bin	Scenario 2.1: Binary	Benign	3,287,196	2,629,757	657,439
				Attack	3,287,196	2,629,757	657,439
				Total	6,574,392	5,259,514	1,314,878
3		CICIDS_Multi	Scenario 2.2: Attack	Benign	160,147	128,118	32,029
				Brute-force	160,784	128,627	32,157
				DDoS	160,939	128,751	32,188
				DoS	160,848	128,678	32,170
				Infiltration	160,674	128,539	32,135
				Portscan	158,804	127,043	31,761
				Total	962,196	769,756	192,440
4	Benchmarking	CICIDS2017	Binary	Benign	540,349	432,279	108,070
				Attack	540,349	432,279	108,070
				Total	1,080,698	864,558	216,140
5		CSE-CIC-IDS2018		Benign	2,746,847	2,197,478	549,369
				Attack	2,746,847	2,197,478	549,369
				Total	5,493,694	4,394,956	1,098,738

TABLE 3. Selected features with their corresponding importance for the different scenarios.

	Number of features	Accuracy
Scenario 1 - Multiclass classification (7 classes)	9	0.97738
Scenario 2.1 - Binary classification (2 classes)	16	0.99499
Scenario 2.2 - Attack classification (6 classes)	7	0.97865

RF, this study has implemented feature selection across the entire dataset, covering all classes simultaneously, to reduce the complexity of the proposed system. Backward Feature Elimination (BFE) was used to determine the optimal number of features that yielded the highest accuracy with the RF classifier, after which these features were ranked using the same classifier. It is noteworthy that accuracy was used as the metric of choice, as the dataset was balanced. Table 3 presents the number of features selected for each case, while Table 4 lists the top features identified along with their corresponding importance. The number of features was not predetermined; instead, the number that yielded the highest accuracy when tested against the classifier was selected. As shown in Table 3, using the full set of 75 features, excluding the class label, does not yield optimal results. Instead, a significantly smaller subset of features is sufficient to produce high-quality results. It is important to note that only the “Destination_Port” feature remained unchanged (i.e., functional feature), as altering it would

affect the functionality of certain attacks, whereas the others (i.e., non-functional features) are modified according to their respective algorithms.

E. MODEL SELECTION

A series of comprehensive experiments was conducted to identify the optimal NIDS model. As illustrated in Fig. 5, four tree-based machine learning classifiers were implemented, including three ensemble methods, alongside two neural network models. For the tree-based models, Tables 5, 6, and 7 detail their default configurations, specifying the number of estimators or trees, their depth, and the splitting criterion. The tree depth for the RF, Adaptive Boosting (AdaBoost), and Decision Tree (DT) models was the only parameter that varied (i.e., was not explicitly specified) based on the purity of the leaves; the tree depth stopped when all leaves were pure. The varying depths of AdaBoost sub-trees were not specified in Tables 5, 6, and 7 due to the independence of each tree or estimator. The default XGBoost settings were maintained, as it rarely requires deeper trees. For the ANN and CNN, hyperparameter tuning was applied to the number of layers (i.e., 1, 2, and 3) and neurons/filters (i.e., 8, 16, 32, 64, 128, and 256). It was observed that accuracy plateaued after two layers for both networks, thus further exploration was not pursued. For the CNN model, the input network features are represented as a one-dimensional vector, enabling 1D convolutional layers to capture dependencies among the selected features. The

TABLE 4. Ranked selected features for each case.

Scenario 1 - Multiclass classification (7 classes)			Scenario 2.1 - Binary classification (2 classes)		
Top 9 Indexes	Feature names	Average Importance	Top 16 Indexes	Feature names	Average Importance
0	Destination_Port	0.268242064	6	Fwd_Packet_Len_Max	0.174197482
34	Fwd_Header_Len	0.148368650	0	Destination_Port	0.143738993
15	Flow_Packets_sec	0.142035914	8	Fwd_Packet_Len_Mean	0.119487762
36	Fwd_Packets_sec	0.118342786	34	Fwd_Header_Len	0.102572759
52	Avg_Packet_Size	0.115859668	52	Avg_Packet_Size	0.061803232
8	Fwd_Packet_Len_Mean	0.075690227	40	Packet_Len_Mean	0.059151469
23	Fwd_IAT_Max	0.074123904	23	Fwd_IAT_Max	0.046099816
17	Flow_IAT_Std	0.029828054	37	Bwd_Packets_sec	0.043022519
22	Fwd_IAT_Std	0.027508733	21	Fwd_IAT_Mean	0.042748085
			15	Flow_Packets_sec	0.040660246
			20	Fwd_IAT_Total	0.034566129
			36	Fwd_Packets_sec	0.034457679
			22	Fwd_IAT_Std	0.032503526
			17	Flow_IAT_Std	0.026312013
			24	Fwd_IAT_Min	0.022435956
			14	Flow_Bytes_sec	0.016242334

Scenario 2.2 - Attack classification (6 classes)		
Top 7 Indexes	Feature names	Average Importance
0	Destination_Port	0.323243002
34	Fwd_Header_Len	0.235063230
36	Fwd_Packets_sec	0.193266123
39	Max_Packet_Len	0.162563736
17	Flow_IAT_Std	0.059240724
73	Idle_Max	0.023686186
43	FIN_Flag_Count	0.002936999

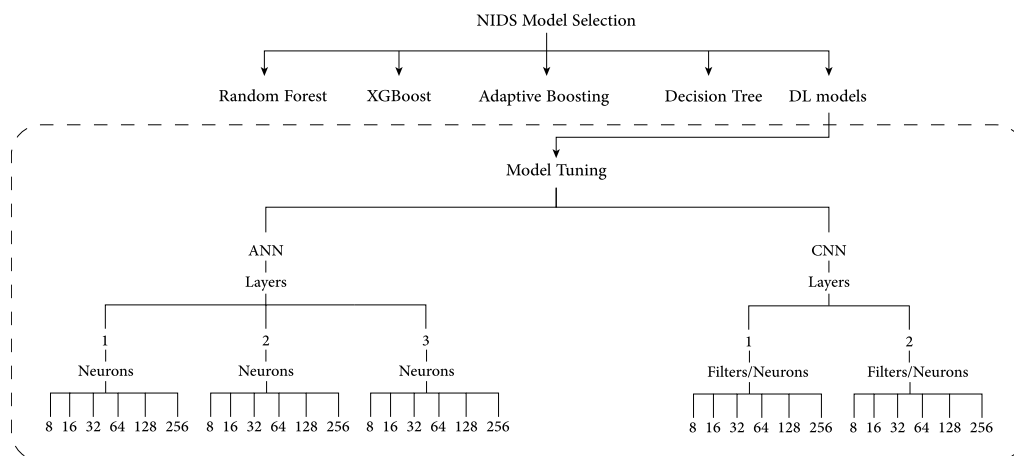
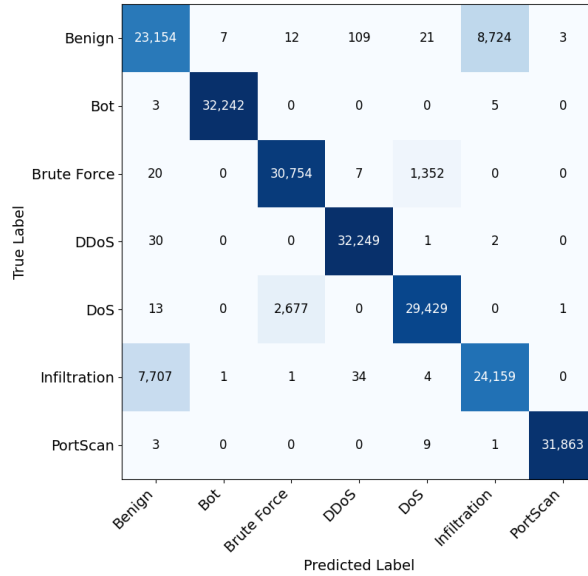


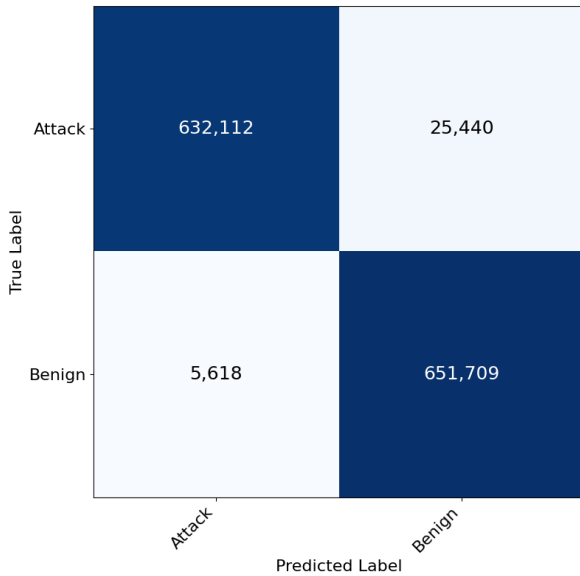
FIGURE 5. Candidate classifiers for the NIDS model selection stage.

extracted feature representations are then passed to MLP for final classification.

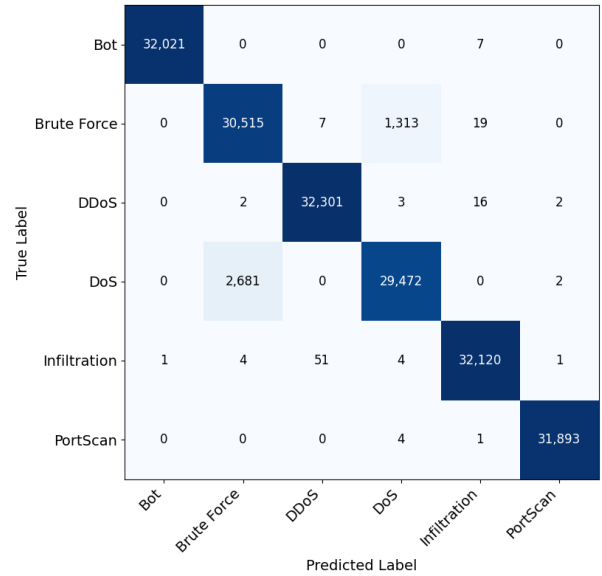
CNNs are suitable for tabular network flow data when features are treated as a one-dimensional sequence, allowing



(a) Scenario 1: Multiclass RF NIDS confusion matrix



(b) Scenario 2.1: Binary XGBoost NIDS confusion matrix



(c) Scenario 2.2: Attack RF NIDS confusion matrix

FIGURE 6. Confusion matrices for the best NIDS for each scenario case.

1D convolutions to capture local dependencies and interactions between features. This enables efficient feature extraction with fewer parameters and improved generalization compared to fully connected models, as supported by prior intrusion detection studies.

Table 5 demonstrates the RF algorithm achieving the highest accuracy at 90.76% for the multiclass network, whereas the one-layer ANN exhibited the lowest performance with an accuracy of 78.97%. The confusion matrix depicted in Fig. 6a reveals a frequent misclassification of infiltration attacks as benign, which aligns with the findings of [35]. Despite the authors statement in [35] regarding the significance of

the total time between two backward-direction packets in detecting infiltration attacks, our evaluation incorporating this feature (Bwd_IAT_Total) still resulted in misclassification, thereby contradicting their claim.

In the binary classifier of the second scenario (i.e., scenario 2.1), XGBoost demonstrated superior performance with an accuracy of 97.64%, as shown in Table 6. Figure 6b illustrates a False Positive Rate (FPR) of 3.76% (i.e., misclassified benign flows) and a False Negative Rate (FNR) of 0.881% (i.e., misclassified attack flows). Although the FPR is relatively low, the more critical FNR remains below 1%. Attacks identified in the first stage proceed to the second

TABLE 5. NIDS results for the tree-based ML classifiers and neural networks for the multiclass classification (scenario 1) on the testing set.

Classifier	#Estimators	Depth	Criterion	Accuracy
RF	100	84	Gini	0.9076
XGBoost	6	6	Info. Gain	0.9024
AdaBoost	50	None	Gini	0.9039
DT	1	86	Gini	0.9017

Classifier	#Layers	#Neurons /Kernels	Loss	Accuracy
ANN	1	64	0.3962	0.7897
	2	64/256	0.3648	0.8813
	3	256/64/64	0.2264	0.8819
CNN	1	256/256	0.2453	0.8759
	2	128/128/128	0.2248	0.8831

TABLE 6. NIDS results for the tree-based ML classifiers and neural networks for the binary classification (scenario 2.1) on the testing set.

Classifier	#Estimators	Depth	Criterion	Accuracy
RF	100	92	Gini	0.9740
XGBoost	100	6	Info. Gain	0.9764
AdaBoost	50	None	Gini	0.9675
DT	1	112	Gini	0.9656

Classifier	#Layers	#Neurons /Kernels	Loss	Accuracy
ANN	1	128	0.1268	0.9644
	2	128/32	0.1038	0.9716
	3	256/64/8	0.1027	0.9719
CNN	1	128/128	0.1031	0.9721
	2	256/32/64	0.0929	0.9754

TABLE 7. NIDS results for the tree-based ML classifiers and neural networks for the attack classification (scenario 2.2) on the testing set.

Classifier	#Estimators	Depth	Criterion	Accuracy
RF	100	45	Gini	0.9786
XGBoost	100	6	Info. Gain	0.9779
AdaBoost	50	None	Gini	0.9785
DT	1	34	Gini	0.9785

Classifier	#Layers	#Neurons /Kernels	Loss	Accuracy
ANN	1	256	0.2803	0.8810
	2	64/64	0.0892	0.9703
	3	256/128/16	0.0855	0.9695
CNN	1	64/64	0.1253	0.9614
	2	16/256/128	0.0762	0.9716

classifier (i.e., scenario 2.2), where RF is again utilized based on the performance as detailed in Table 7. The attack classification accuracy improves compared to the first scenario. Figure 6c indicates that the infiltration attack is no longer misclassified due to the separation of benign from individual attack classifications. However, a minor portion of DoS flows are misclassified as brute-force.

In summary, the second scenario demonstrates superior performance to the first, achieving accuracies of 97.64% and

97.86%, compared to the 90.76% across all seven classes in the first scenario. Thus, the second scenario will be implemented for the NIDS. Furthermore, only the initial binary classifier will be used for retraining, as the primary objective is to distinguish between adversarial attacks and benign samples, a task handled by the first-line defense network. Given that white-box algorithms depend on gradients, a two-layer CNN will be used to generate samples and selected as the victim NIDS, as they require identical models. Based on the tuning results, 256 kernels will be used in the first convolutional layer, 32 kernels in the second convolutional layer, and 64 neurons in the third MLP layer. Meanwhile, for the black-box approach, XGBoost will be utilized to generate the samples.

F. ADVERSARIAL RETRAINING

After selecting features and models, adversarial samples were generated using the entire attack training set. Notably, the CW algorithm yielded a significantly lower number of samples, specifically 29,248 adversarial instances for the combined dataset (i.e., CICIDS_Bin). Consequently, all algorithms were configured to produce an equivalent number of samples based on the CW algorithm across all datasets. To construct the final adversarial training and testing sets, the number of normal attack samples was equivalent to the number of adversarial instances, while the number of benign samples was set to be equal to the combined total number of adversarial and normal attack samples, as presented in Table 8. Given the general difficulty in acquiring attack data, the proportion of attack data was intentionally limited compared to benign data to better reflect realistic conditions. An 80:20 split was implemented for training and testing, and a 90:10 split for training and validation. During the training phase, we combined all black-box samples and similarly for the white-box samples. However, for testing, we evaluated the model against each algorithm separately.

IV. EXPERIMENTS

A. EXPERIMENTAL SETUP

Dataset preprocessing, model selection, training, testing, and adversarial retraining were conducted on a macOS machine equipped with an 8-core Intel Core i9 processor and 64 GB of RAM. Computationally intensive tasks, including feature selection and adversarial attack generation, were performed on Ubuntu 24.04.1 LTS machines with 16-core processor and 32 GB of RAM. Most adversarial sample generation algorithms completed within 30 minutes. However, the CW and JSMA attacks required significantly longer execution times, where they took from 2 to 3 days. All experiments were implemented in Python 3.8.5. The CNN model was trained using the Adam optimizer, with an initial learning rate of 10^{-3} for the base NIDS (i.e., prior to adversarial retraining).

To retrain our CNN NIDS model on the adversarial dataset, we experimented with the last layer of our NIDS (i.e., MLP) and the last two layers (i.e., convolution and MLP). For the

TABLE 8. Training and testing sample distribution for the datasets.

	CICIDS_Bin (per attack)				CICIDS_Bin (all 12 attacks)			
	Adversarial	Attack	Benign	Total	Adversarial	Attack	Benign	Total
Training	20,900	20,900	41,800	83,600	250,800	250,800	501,600	1,003,200
Validation	2,418	2,418	4,836	9,672	29,016	29,016	58,032	116,064
Testing	5,930	5,930	11,860	23,720	71,160	71,160	142,320	284,640

	CICIDS2017 (per attack)				CICIDS2017 (all 12 attacks)			
	Adversarial	Attack	Benign	Total	Adversarial	Attack	Benign	Total
Training	10,016	10,016	20,032	40,064	120,192	120,192	240,384	480,768
Validation	1,000	1,000	2,000	4,000	12,000	12,000	24,000	48,000
Testing	2,504	2,504	5,008	10,016	30,048	30,048	60,096	120,192

	CSE-CIC-IDS2018 (per attack)				CSE-CIC-IDS2018 (all 12 attacks)			
	Adversarial	Attack	Benign	Total	Adversarial	Attack	Benign	Total
Training	6,430	6,430	12,860	25,720	77,160	77,160	154,320	308,640
Validation	728	728	1,456	2,912	8,736	8,736	17,472	34,944
Testing	1,728	1,728	3,456	6,912	20,736	20,736	41,472	82,944

TABLE 9. Best NIDS retraining configurations.

Retraining Condition	Dataset	#Retrained Layers	Learning rate	Epochs	Training Acc.	Testing Acc.
White-box	CICIDS_Bin	1 Layer	10^{-6}	100	86.106%	89.227%
White-box	CICIDS2017	1 Layer	10^{-6}	200	92.525%	93.406%
White-box	CSE-CIC-IDS2018	1 Layer	10^{-6}	200	85.102%	84.546%
Black-box	CICIDS_Bin	2 Layers	10^{-7}	100	73.942%	76.503%
Black-box	CICIDS2017	1 Layer	10^{-6}	200	89.629%	89.069%
Black-box	CSE-CIC-IDS2018	1 Layer	10^{-6}	200	82.687%	84.172%
All adversarial	CICIDS_Bin	2 Layers	10^{-7}	300	94.523%	94.951%
All adversarial	CICIDS2017	1 Layer	10^{-6}	200	86.527%	89.159%
All adversarial	CSE-CIC-IDS2018	1 Layer	10^{-6}	200	87.815%	90.373%

learning rate, we tested 10^{-6} and 10^{-7} since larger rates lead to unstable learning. Table 9 presents the network configurations, along with the training and testing accuracies for the best configuration under the same retraining condition and dataset. The batch size was set to 32, and the stopping criteria (i.e., early stopping) is activated if the training accuracy does not change for 10 consecutive epochs.

B. COMPUTATIONAL COMPLEXITY

The computational time complexity of the proposed two-layer CNN-based NIDS is approximated using (2), which uses a simplified one-dimensional formulation for the CNN [36]. The first term approximates the cost of the convolutional layers in terms of the number of input channels from the previous layer k_{n-1} , kernel size s_n , number of filters f_n , and output length l_n . Although [36] incorporates both the learning rate and batch size into the formulation, we exclude them as the complexity is defined per iteration and depends only on the model architecture.

$$O\left(\sum_{n=1}^d k_{n-1} \cdot s_n \cdot f_n \cdot l_n\right) + O(l_n \cdot f_n) + O\left(\sum_{l=1}^f N_{in}^l \cdot N_{out}^l\right) \quad (2)$$

We would also note that the authors in [36] are directly multiplying the computational cost by the learning rate which leads to an incorrect interpretation, where smaller learning rates would require fewer operations, contradicting the fact that lower learning rates typically require more iterations and thus greater overall computational effort.

The second term represents the computational cost of the flatten layer, which performs a linear transformation of the input into a flattened one channel vector. The final term corresponds to the fully connected layers, where N_{in} and N_{out} present the input and output dimensions of each layer, respectively.

The proposed CNN requires approximately 438,912 operations across the two convolutional layers, flatten layer, and fully connected layers for an input size of (16,1). The execution time is reported in Table 10. The inference time across all models and datasets ranged between 23 and 24 milliseconds, computed by averaging the runtime over 100 samples per model, indicating that the proposed approach is suitable for real-time applications.

C. RESULTS AND EVALUATION

To assess the training methodologies, we conducted a series of experiments for each approach, as outlined below:

TABLE 10. Training time for each model and dataset.

Training model	Dataset	#Training samples	#Trainable parameters	Training time	Avg. training time per epoch
Initial 2-layer 1D-CNN NIDS	CICIDS_Bin	5,259,513	58,594	4hrs.	12 mins (20 epochs)
XGBoost black-box NIDS	CICIDS_Bin	5,259,513	100 estimators, max depth 6	24.8 sec.	—
White-box CNN retraining	CICIDS_Bin	559,632	32,962	32 min.	19.4 sec. (100 epochs)
White-box CNN retraining	CICIDS2017	264,384	32,962	32 min.	9.5 sec. (200 epochs)
White-box CNN retraining	CSE-CIC-IDS2018	171,792	32,962	35 min.	7.5 sec. (200 epochs)
Black-box CNN retraining	CICIDS_Bin	559,632	57,570	1 hr 10 min.	42.1 sec. (100 epochs)
Black-box CNN retraining	CICIDS2017	264,384	32,962	38 min.	11.3 sec. (200 epochs)
Black-box CNN retraining	CSE-CIC-IDS2018	171,792	32,962	28 min.	8.3 sec. (200 epochs)
All-adversarial attacks CNN retraining	CICIDS_Bin	1,119,264	57,570	8 hrs.	9.7 sec. (300 epochs)
All-adversarial attacks CNN retraining	CICIDS2017	528,768	32,962	1 hr 10 min.	21 sec. (200 epochs)
All-adversarial attacks CNN retraining	CSE-CIC-IDS2018	343,584	32,962	39 min.	12.7 sec. (200 epochs)

- Evaluation against the original dataset to determine the methods performance post-adversarial retraining in normal scenarios.
- Assessment against all white-box, all black-box, and all adversarial attacks to test overall detection performance under diverse conditions.
- Examination of each adversarial attack method individually to assess the robustness of each attack algorithm in evading detection.

As depicted in Fig. 7, the original training method exhibited optimal performance in detecting only normal attacks and benign samples. Conversely, the white-box and black-box attacks demonstrated opposite trends. The white-box attacks showed a decline in performance when tested against adversarial samples compared to clean samples, whereas the black-box attacks performed more effectively against adversarial samples than against normal samples. As previously noted, black-box attack samples attempt to evade detection by using a completely different model for attack generation, thereby enhancing their sophistication in terms of attack strategy and defense model robustness. Finally, the all-attacks retrained model proves to be the most efficient as it maintained stable performance across all conditions.

Upon examining Table 11, it is evident that the most challenging white-box attack to detect is the BIM, whereas the HopSkipJump is the most difficult among black-box attacks. Across all datasets, performance, as measured by the F1-score, is consistent for each class (i.e., benign and attack), thus we used the average score. White-box attacks are easily detected by all retraining methods because they resemble normal attacks, with only slight deviations that position them between normal and adversarial behavior, as revealed by examining the original network gradients. Therefore, the black-box method, retrained on both normal and more pronounced adversarial attack samples, maintained comparable performance to white-box attacks across most white-box algorithms. Furthermore, the white-box retraining approach achieved the highest F1-score for detecting original samples, aligning with our previous statement. In contrast, a significant decline in performance is observed for the

TABLE 11. Average F1-scores under different training and testing settings for different classes.

Test Attacks / Train Data	White-box attacks					
	PGD	FGSM	BIM	CW	DeepFool	JSMA
Original	0.58	0.59	0.58	0.61	0.61	0.61
White-box	0.89	0.87	0.85	0.92	0.91	0.90
Black-box	0.86	0.89	0.81	0.89	0.94	0.92
All adversarial	0.90	0.91	0.87	0.92	0.94	0.93

Test Attacks / Train Data	Black-box attacks					
	ZOO	Bound-ary	Point-wise	HopSkip-Jump	OPT	GA
Original	0.59	0.58	0.58	0.56	0.61	0.62
White-box	0.86	0.80	0.80	0.70	0.87	0.89
Black-box	0.89	0.87	0.94	0.83	0.90	0.93
All adversarial	0.89	0.84	0.91	0.78	0.90	0.90

Test Attacks / Train Data	All white box	All black box	All adversarial attacks	Original clean data
Original	0.59	0.59	0.59	0.98
White-box	0.89	0.82	0.86	0.96
Black-box	0.88	0.89	0.89	0.78
All adversarial	0.91	0.87	0.89	0.93

black-box retraining method on the original samples, reflecting the dissimilarity between original and adversarial attack samples, which arises from differences in the NIDS architectures used.

Ultimately, the proposed all-attacks adversarial retraining method demonstrated superior robustness by maintaining stable performance across original and all-attack conditions and algorithms, except for the HopSkipJump attacks, which require further investigation (e.g., generating additional samples). The generalized high-performing results are attributed to a broader spectrum of unseen attacks during training rather than to a subset generated under the same adversarial condition (i.e., white- or black-box). Overall, a comparison between the original NIDS and the white-box, black-box, and all adversarial retraining approaches reveals increases in the average F1-score for adversarial attack detection of 0.27, 0.30, and 0.30, respectively, with an F1-score of 0.59 prior to

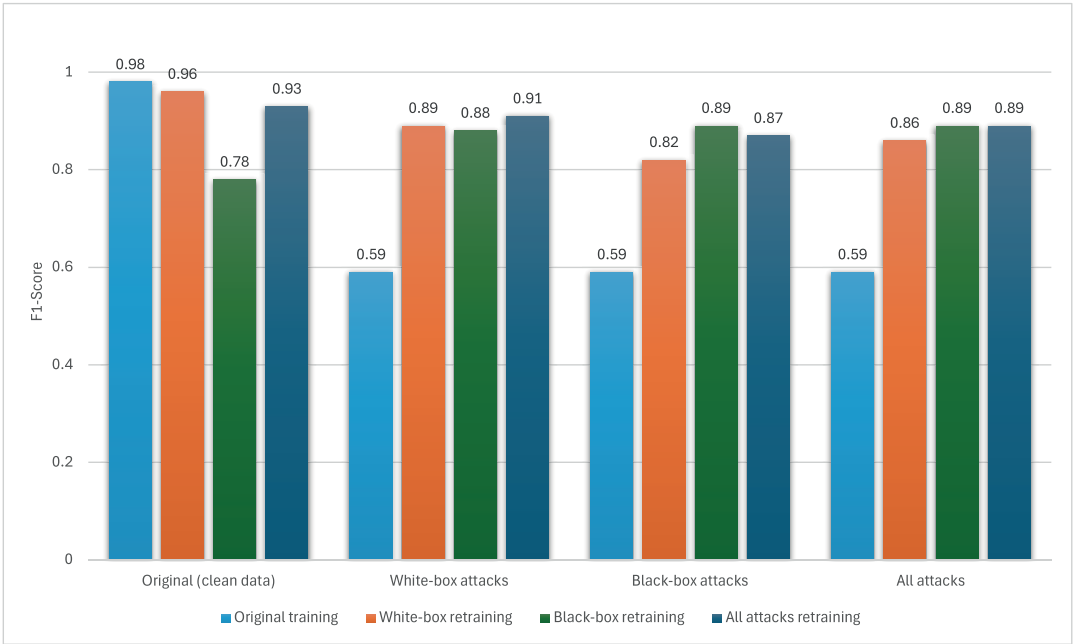


FIGURE 7. The average F1-scores on the testing set for the default and different adversarial training approaches for the CICIDS_Bin dataset.

TABLE 12. Performance accuracy comparison between our proposed NIDS with the retraining approaches and existing work on the CICIDS2017 dataset in a white-box testing setting. Numbers in parentheses represent the relative accuracy improvement, calculated as the difference between the highest accuracy achieved by the proposed model and the highest accuracy reported by the comparison method for the same attack.

CICIDS2017						
Model	Trained/Tested Attacks	FGSM	PGD	CW	DeepFool	Balanced
Parallel DNN-IDS [37]	Balanced: FGSM + PGD + CW + DeepFool	99.87% (+2.27%)	99.84% (+4.53%)	88.38%	91.69%	95.99%
3-Layer CNN (Proposed)	White-Box Retraining	94.82%	92.27%	92.80%	91.69%	92.89%
	Black-Box Retraining	93.56%	90.35%	85.37%	93.22%	90.63%
	All Attacks Retraining	97.60%	95.31%	95.58% (+7.02%)	97.37% (+5.68%)	96.47% (+0.48%)

TABLE 13. Performance accuracy comparison between our proposed NIDS with the retraining approaches and existing work on the CSE-CIC-IDS2018 dataset in a black-box testing setting. Numbers in parentheses represent the relative accuracy improvement, calculated as the difference between the highest accuracy achieved by the proposed model and the highest accuracy reported by the comparison method for the same attack.

CSE-CIC-IDS2018					
Model	Trained/Tested Attacks	FGSM	BIM	DeepFool	JSMA
MLP [38]	FGSM	92.4%	84.6%	74.1%	86.7%
	BIM	74.2%	93.1%	85.3%	77.3%
	DeepFool	62.4%	75.6%	92.5%	69.2%
	JSMA	74.5%	81.2%	74.3%	91.4%
	C _{at} : FGSM + BIM + DeepFool + JSMA	92.7%	93.4%	93.1%	92.3% (+5.6%)
3-Layer CNN (Proposed)	White-Box Retraining	85.1%	87.1%	86.5%	85.7%
	Black-Box Retraining	91.4%	91.5%	94.2% (+1.1%)	81.4%
	All Attacks Retraining	96.4% (+3.7%)	96.3% (+2.9%)	92.5%	86.7%

adversarial training and 0.89 following adversarial retraining. This highlights the importance of adversarial retraining.

We have evaluated our retraining approaches against two existing methodologies, as illustrated in Tables 12 and 13,

utilizing each dataset independently rather than our combined dataset (i.e., CICIDS_Bin). The comparison was conducted with [37] and [38], as these studies fulfill our selection criteria, specifically the use of same datasets and feature set. To the best of our knowledge, these were the only works meeting these criteria, as other studies either used different datasets or combined the selected datasets with others.

For the CICIDS2017 dataset, the retraining approaches were assessed on four white-box algorithms, as used in [37], along with a balanced testing set comprising the four algorithms combined. As demonstrated in Table 12, despite the complexity of their proposed parallel DNN, which incorporates several detectors (i.e., ensemble network), our simple two-layer CNN with an all-attacks retraining approach (i.e., retraining only the final MLP layer) achieved overall higher accuracy which proves the efficiency and effectiveness of our model.

In our evaluation of the CSE-CIC-IDS2018 dataset, we compare our results with those of [38], which used four white-box adversarial algorithms trained using a black-box method. To ensure a fair comparison, we implemented the same methodology exclusively on the testing set. Additionally, we utilized a different NIDS to generate adversarial testing samples using the 3-layer ANN model as detailed in Table 6. The authors in [38] performed adversarial retraining on each algorithm individually and collectively across their four algorithms. As indicated in Table 13, the CNN with retraining on all adversarial attacks demonstrated superior detection accuracy for the FGSM and BIM attacks, while the black-box retraining approach exhibited marginally better performance on the DeepFool algorithm. The only limitation was observed with the JSMA attack; however, it is worth noting that our proposed method offers a comprehensive approach that addresses 12 attacks, rather than only 4. It is intuitive that a model trained and tested on the same small subset of attacks would perform better than a more comprehensive, generalized model. By covering a wider range of attacks, our method is less likely to misclassify an unseen threat. This makes our intrusion detection system accurate in a testing setting and resilient and adaptable in real-world.

V. CONCLUSION

In this study, we developed a proactive and resilient NIDS using three adversarial retraining methods. Our approach integrated six black-box and six white-box adversarial samples, alongside an all-attacks retraining scheme, to ensure robustness against a diverse array of threats. We conducted a comprehensive evaluation using four tree-based classifiers, including three ensemble models and two neural networks, and trained and tested them on the widely used CICIDS2017 and CSE-CIC-IDS-2018 datasets. These datasets encompassed six main distinct categories of network attacks. Our findings indicate that a two-stage NIDS, in particular, demonstrated strong performance, with a limited subset of features proving sufficient to achieve high performance. The most

effective machine learning algorithm, XGBoost, was used to generate black-box attacks, while the CNN was used for white-box attack generation, which requires gradient information. The experimental results demonstrate that while some models might perform better against a small subset of attacks, retraining them with adversarial examples from a broad spectrum of adversarial attack types significantly enhances their ability to generalize and detect new, unseen threats, which is an essential quality for any security solution deployed in real-world scenarios. Additionally, simpler models are sufficient to provide a robust defense system. Furthermore, as part of our future work, we plan to enhance our NIDS to autonomously generate adversarial samples using GANs, thereby further extending the diversity of threats encountered. We intend to implement a secondary cleaning module that denoises inputs, particularly when the prediction confidence is ambiguous (i.e., near the 50% prediction threshold). These enhancements will advance cybersecurity by enabling automated, self-improving intrusion detection systems.

REFERENCES

- [1] C. Group, *Cyberthreat Defense Report*. Accessed: Apr. 7, 2026. [Online]. Available: <https://cyberedgegroup.com/cdr/>
- [2] M. Monte, *Network Attacks and Exploitation: A Framework*. Indianapolis, IN, USA: Wiley, 2015.
- [3] X. Zhang, J. Wang, and S. Zhu, "Dual generative adversarial networks based unknown encryption ransomware attack detection," *IEEE Access*, vol. 10, pp. 900–913, 2022.
- [4] *KDD Cup 1999 Data*. Accessed: Feb. 7, 2023. [Online]. Available: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [5] V. Heydari and K. Nyarko, "Enhancing adversarial robustness in network intrusion detection: A novel adversarially trained neural network approach," *Electronics*, vol. 14, no. 16, p. 3249, Aug. 2025.
- [6] K. Barik and S. Misra, "A comprehensive defense approach of deep learning-based NIDS against adversarial attacks," *Multimedia Tools Appl.*, vol. 84, no. 31, pp. 1–47, Jul. 2025.
- [7] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Oct. 2019, pp. 1–8.
- [8] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6.
- [9] L. Li et al., "Towards effective black-box attacks on DoH tunnel detection systems," *Comput. Netw.*, vol. 270, Oct. 2025, Art. no. 111524.
- [10] Z. Mao, Y. Lu, Z. Cheng, and K. Chen, "Exploring transferable adversarial attacks for deep learning-based network intrusion detection," *J. New. Comput. Appl.*, vol. 242, Oct. 2025, Art. no. 104255.
- [11] B. Tafreshian and S. Zhang, "A defensive framework against adversarial attacks on machine learning-based network intrusion detection systems," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2024, pp. 2436–2441.
- [12] N. Alhussien and A. Aleroud, "AdvPurRec: Strengthening network intrusion detection with diffusion model reconstruction against adversarial attacks," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2024, pp. 1638–1646.
- [13] M. Huang, Y. Lin, N. Li, X. Chen, and E. Bertino, "CARD: robustness-preserving transfer learning for network intrusion detection via contrastive adversarial representation distillation," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 5134–5151, Sep. 2025.
- [14] M. E. Shehaby and A. Matrawy, "Introducing perturb-ability score (PS) to enhance robustness against problem-space evasion adversarial attacks on flow-based ML-NIDS," 2024, *arXiv:2409.07448*.
- [15] U. A. Ali, K. Dogra, and S. Sharma, "White-box adversarial exploitation of NIDS: Insights from FGSM, PGD, and C&W," in *Proc. 2nd Int. Conf. Comput. Intell., Commun. Technol. Netw. (CICITN)*, Feb. 2025, pp. 668–673.

- [16] P. Dadkhah, S. Ahmadi, and M. R. Aref, "GenAtt NIDS attack: A practical generative adversarial attack against network intrusion detection systems," in *Proc. 29th Int. Comput. Conf., Comput. Soc. Iran (CSICC)*, Iran, Feb. 2025, pp. 1–7.
- [17] N. Liu, H. Wu, Y. Zhang, and C. Wang, "Adversarial attacks against black-box network intrusion detection based on heuristic algorithm," in *Proc. 10th Int. Conf. Comput. Commun. (ICCC)*, Dec. 2024, pp. 1954–1958.
- [18] A. Grini, O. Taheri, B. El Khamlichi, and A. El Fallah-Seghrouchni, "Constrained network adversarial attacks: Validity, robustness, and transferability," in *Proc. 21st Int. Conf. Distrib. Comput. Smart Syst. Internet Things (DCOSS-IoT)*, Jun. 2025, pp. 771–776.
- [19] W. Du, J. Xue, X. Yang, W. Guo, D. Gu, and W. Han, "Transfformer: A novel transformer-based framework to generate evasive malicious traffic," *Knowledge-Based Syst.*, vol. 319, Jun. 2025, Art. no. 113546.
- [20] S. Okada et al., "Xai-driven black-box adversarial attacks on network intrusion detectors," *Int. J. Inf. Secur.*, vol. 24, no. 3, pp. 1–15, Jun. 2025.
- [21] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 39–57.
- [22] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.
- [23] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [24] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial Intelligence Safety and Security*, 2018, pp. 99–112.
- [25] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2574–2582.
- [26] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Mar. 2016, pp. 372–387.
- [27] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, Dallas, TX, USA, Nov. 2017, pp. 15–26.
- [28] W. Brendel, J. Rauber, and M. Bethge, "Decision-based adversarial attacks: Reliable attacks against black-box machine learning models," 2017, *arXiv:1712.04248*.
- [29] L. Schott, J. Rauber, M. Bethge, and W. Brendel, "Towards the first adversarially robust neural network model on mnist," 2018, *arXiv:1805.09190*.
- [30] J. Chen, M. I. Jordan, and M. J. Wainwright, "Hopskipjumpattack: A query-efficient decision-based attack," in *Proc. IEEE Symp. Secur. Privacy*, Mar. 2020, pp. 1277–1294.
- [31] M. Cheng, T. Le, P.-Y. Chen, J. Yi, H. Zhang, and C.-J. Hsieh, "Query-efficient hard-label black-box attack: An optimization-based approach," 2018, *arXiv:1807.04457*.
- [32] E. Alhajjar, P. Maxwell, and N. Bastian, "Adversarial machine learning in network intrusion detection systems," *Expert Syst. Appl.*, vol. 186, Dec. 2021, Art. no. 115782.
- [33] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
- [34] *CSE-CIC-IDS2018 on AWS*. Accessed: Dec. 3, 2024. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2018.html>
- [35] C. Zhang, X. Costa-Perez, and P. Patras, "Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1294–1311, Jun. 2022.
- [36] B. Shah and H. Bhavsar, "Time complexity in deep learning models," *Proc. Comput. Sci.*, vol. 215, pp. 202–210, Jan. 2022.
- [37] I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza, and W. Mees, "TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems," *Future Gener. Comput. Syst.*, vol. 138, pp. 185–197, Jan. 2023.
- [38] J. Wang, J. Pan, I. AlQerm, and Y. Liu, "Def-IDS: An ensemble defense mechanism against adversarial attacks for deep learning-based network intrusion detection," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jul. 2021, pp. 1–9.